Merging Linked Lists

AlgoExpert · Medium · Linked Lists

The problem

- Two lists may converge into a shared tail; return the first shared node or null.
- Two pointers that switch lists meet at the junction after at most n+m steps.

Intuition

- Two pointers that switch to the other list after finishing travel the same distance and meet at the junction.

Brute force vs optimal

- Brute force: for each A node, scan B → O(n·m)
- Hash set of A nodes → O(n + m) time, O(n) space
- Two pointers switching lists → O(1) space

Approach

- 1. Advance both pointers
- 2. When one ends, jump to the other list's head
- 3. They meet at the shared node or both null

Worked example

list A



list B (shares 1 → 4)



Both lists share the tail 1 → 4

Worked example

```
p walks A then B: step 0 → 2

q walks B then A: step 0 → 8
```

Step 0

Worked example

```
p walks A then B: step 2 → 1

q walks B then A: step 2 → 1
```

Step 2

Worked example

```
p walks A then B: step 5 → 8

q walks B then A: step 5 → 2
```

Step 5

Worked example

```
p walks A then B: step 7 → 1

q walks B then A: step 7 → 1
```

Step 7

Worked example

```
both travel lenA + lenB → they meet at the shared node

meet at node 1 (the shared one)
```

They meet at the shared node

Java solution

```java
public class MergingLinkedLists {
    public static LinkedListNode mergingLinkedLists(LinkedListNode a, LinkedListNode b) {
        LinkedListNode p = a, q = b;
        while (p != q) { p = p == null ? b : p.next; q = q == null ? a : q.next; }
        return p;
    }
}
```

Complexity

- Time: O(n + m)
- Space: O(1)

Edge cases & pitfalls

- Compare node identity, not value
- No intersection → both reach null together
- Different lengths are fine

Interview tips & follow-ups

- Explain why the distances equalise
- Alternative: compute lengths and align

Recap

- Two pointers that switch to the other list after finishing travel the same distance and meet at the junction.
- Complexity: O(n + m)